MODULE 3 – CLASSES AND METHODS

1. Explain Class in java using syntax and example.

A class is a *template* for an object, and an object is an *instance* of a class. Because an object is an instance of a class, you will often see the two words *object* and *instance* used interchangeably.

The General Form of a Class

When you define a class, you declare its exact form and nature. You do this by specifying the data that it contains and the code that operates on that data. While very simple classes may contain only code or only data, most real-world classes contain both. As you will see, a class' code defines the interface to its data.

A class is declared by use of the **class** keyword. The classes that have been used up to this point are actually very limited examples of its complete form. Classes can (and usually do) get much more complex. A simplified general form of a **class** definition is shown here:

Syntax

class *classname* {
type instance-variable1;
type instance-variable2;
type instance-variableN;
type methodname1(*parameter-list*) {
// body of method
}
type methodname2(*parameter-list*) {
// body of method
}
type methodnameN(*parameter-list*) {
// body of method
}
}

- The data, or variables, defined within a **class** are called *instance variables.*
- The code is contained within *methods.* Collectively, the methods and variables defined within a class are called *members* of the class.
- In most classes, the instance variables are acted upon and accessed by the methods defined for that class. Thus, as a general rule, it is the methods that determine how a class' data can be used.
- Variables defined within a class are called instance variables because each instance of the class (that is, each object of the class) contains its own copy of these variables. Thus, the data for one object is separate and unique from the data for another.
- All methods have the same general form as **main( )**, which we have been using thus far. However, most methods will not be specified as **static** or **public**. Notice that the general form of a class does not specify a **main( )** method. Java classes do not need to have a **main( )** method. You only specify one if that class is the starting point for your program. Further, applets don't require a **main( )** method at all.

Example

```
class Box {
double width; double height; double depth;
}
```

ACET

```
// This class declares an object of type Box. class BoxDemo {
public static void main(String args[]) {
Box mybox = new Box(); double vol;
// assign values to mybox's instance variables mybox.width = 10; mybox.height = 20; mybox.depth = 15;
// compute volume of box
vol = mybox.width * mybox.height * mybox.depth;
System.out.println("Volume is " + vol);
}
}
```

output:
Volume is 3000.0

2. Explain methods using an example.

This is the general form of a method:

```
type name(parameter-list) {
 // body of method
}
```

- Here, *type* specifies the type of data returned by the method. This can be any valid type,
- including class types that you create. If the method does not return a value, its return type must be **void**. The name of the method is specified by *name.* This can be any legal identifier other than those already used by other items within the current scope.
- The *parameter-list* is a sequence of type and identifier pairs separated by commas. Parameters are essentially variables that receive the value of the *arguments* passed to the method when it is called. If the method has no parameters, then the parameter list will be empty.
- Methods that have a return type other than **void** return a value to the calling routine using the following form of the **return** statement:
- return *value*;
- Here, *value* is the value returned.
- In the next few sections, you will see how to create various types of methods, including those that take parameters and those that return values.

Example

```
class Box {
double width; double height; double depth;
// display volume of a box void volume() {
System.out.print("Volume is ");
System.out.println(width * height * depth);
}
}
class BoxDemo3 {
public static void main(String args[]) {
Box mybox1 = new Box();
Box mybox2 = new Box();
// assign values to mybox1's instance variables mybox1.width = 10; mybox1.height = 20; mybox1.depth = 15;
```

ACET

/* assign different values to mybox2's instance variables */ mybox2.width = 3; mybox2.height = 6; mybox2.depth = 9;
// display volume of first box mybox1.volume();
// display volume of second box mybox2.volume();
}
}
Output
Volume is 3000.0
Volume is 162.0

3. What are constructors ? Explain two types of constructors.

A constructor initializes an object immediately upon creation. It has the same name as the class in which it resides and is syntactically similar to a method. Once defined, the constructor is automatically called immediately after the object is created, before the new operator completes. Constructors look a little strange because they have no return type, not even void.

when you allocate an object, you use the following general form:
class-var = new classname( );
for example:
Box mybox1 = new Box();
When you do not explicitly define a constructor for a class, then Java creates a default constructor for the class. The default constructor automatically initializes all instance variables to zero. The default constructor is often sufficient for simple classes, but it usually won't do for more sophisticated ones. Once you define your own constructor, the default constructor is no longer used.
Example

```
class Box {
double width; double height; double depth;
Box() {
System.out.println("Constructing Box");
width = 10;
height = 10;
depth = 10;
}
double volume() {
return width * height * depth;
}
}
class BoxDemo6 {
public static void main(String args[]) {
Box mybox1 = new Box();
Box mybox2 = new Box();
double vol;
 vol = mybox1.volume();
 System.out.println("Volume is " + vol);
vol = mybox2.volume();
System.out.println("Volume is " + vol);
}
}
```

Output:

```
Constructing Box
Constructing Box
Volume is 1000.0
Volume is 1000.0
```

Parameterized Constructors

A constructor that accepts arguments to initialize an objects attributes with a specific values.

```
class Box {
double width; double height; double depth;
Box(double w, double h, double d) {
width = w;
height = h;
depth = d;
}
double volume() {
return width * height * depth;
}
}class BoxDemo7 {
public static void main(String args[]) {
Box mybox1 = new Box(10, 20, 15);
Box mybox2 = new Box(3, 6, 9);
double vol;
vol = mybox1.volume();
System.out.println("Volume is " + vol);
vol = mybox2.volume();
System.out.println("Volume is " + vol);
}
}
```

```
Volume is 3000.0
Volume is 162.0
```

4. Write a program to perform Stack operation using proper class and methods

```
class TestStack {
public static void main(String args[]) {
Stack mystack1 = new Stack();
Stack mystack2 = new Stack();
for(int i=0; i<10; i++)
mystack1.push(i);
for(int i=10; i<20; i++)
mystack2.push(i);
System.out.println("Stack in mystack1:");
for(int i=0; i<10; i++)
System.out.println(mystack1.pop());
System.out.println("Stack in mystack2:");
for(int i=0; i<10; i++)
System.out.println(mystack2.pop());
}
}
```

Output

```
Stack in mystack1:
9
8
7
6
5
4
3
2
1
0 Stack in
mystack2:
19
18
17
16
15
14
13
12
11
10
```

5. Explain memory allocation and use of garbage collector in JAVA

Since objects are dynamically allocated by using the new operator, you might be wondering how such objects are destroyed and their memory released for later reallocation. In some languages, such as C++, dynamically allocated objects must be manually released by use of a delete operator.

Java takes a different approach; it handles deallocation for you automatically. The technique that accomplishes this is called garbage collection. It works like this: when no references to an object exist, that object is assumed to be no longer needed, and the memory occupied by the object can be reclaimed.

There is no explicit need to destroy objects as in C++. Garbage collection only occurs sporadically (if at all) during the execution of your program.

It will not occur simply because one or more objects exist that are no longer used. Furthermore, different Java run-time implementations will take varying approaches to garbage collection, but for the most part, you should not have to think about it while writing your programs.

6. Write a JAVA program demonstrating Method overloading.

Method overloading allows multiple methods in the same class to share the same name but di er in:

- Number of parameters
- Type of parameters
- Order of parameters

It enhances readability and improves code maintainability by using a single method name for variations of the same operation.

ACET

```
class OverloadDemo{
void test(){
System.out.println("No Parameters");
}
void test(int a){
System.out.println("a:"+a);
}
void test(int a, int b) {
System.out.println("a and b: " + a + " " + b);
}

double test(double a) {
System.out.println("double a: " + a);
return a*a;
}
}
class Overload {
public static void main(String args[]) {
OverloadDemo ob = new OverloadDemo();
double result;
ob.test();
ob.test(10);
ob.test(10, 20);
result = ob.test(123.25);
System.out.println("Result of ob.test(123.25): " + result);
}
}
```

```
No Parameters
a:10
a and b: 10 20
double a: 123.25
Result of ob.test(123.25): 15190.5625
```

6. Explain the use of this in JAVA with an example.

In Java, the this keyword is a special reference used within an instance method or constructor to refer to the current object. It helps di erentiate between instance variables and parameters when they share the same name, and it provides a way to invoke other constructors within the same class.

Box(double w, double h, double d) { this.width = w; this.height = h; this.depth = d;

```
}
class Studentdetails {
    String name;
    int age;

    Studentdetails(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void display() {
        System.out.println("Name: " + this.name + ", Age: " + this.age);
    }
}

public class Student {
    public static void main(String[] args) {
        Studentdetails s1 = new Studentdetails("Alice", 20);
        s1.display();
    }
}
```

```
Name: Alice, Age: 20
```

7. Define recursion. Write a recursive program to find nth Fibonacci number.

Java supports recursion. Recursion is the process of defining something in terms of itself. As it relates to Java programming, recursion is the attribute that allows a method to call itself.

A method that calls itself is said to be recursive.

The classic example of recursion is the computation of the factorial of a number. The factorial of a number N is the product of all the whole numbers between 1 and N.

For example, 3 factorial is 1 × 2 × 3, or 6. Here is how a factorial can be computed by use of a recursive method:

```
class Factorial {
int fact(int n) {
int result;
if(n==1) return 1;
result = fact(n-1) * n;
return result;
}
}
class Recursion {
public static void main(String args[]) {
Factorial f = new Factorial();
System.out.println("Factorial of 3 is " + f.fact(3));
System.out.println("Factorial of 4 is " + f.fact(4));
System.out.println("Factorial of 5 is " + f.fact(5));
}
}
```

```
Factorial of 3 is 6
Factorial of 4 is 24
Factorial of 5 is 120
```

8. Explain the various access specifiers in Java

member can be accessed is determined by the access specifier that modifies its declaration. Java supplies a rich set of access specifiers. Some aspects of access control are related mostly to inheritance or packages. Java's access specifiers are public, private, and protected. Java also defines a default access level.

protected applies only when inheritance is involved.

Public: When a member of a class is modified by the public specifier, then that member can be accessed by any other code.

Private: When a member of a class is specified as private, then that member can only be accessed by other members of its class.

When no access specifier is used, then by default the member of a class is public.

Example for public access specifier:

```
class Test {
 int a;
 int b;
 int c;
void setc(int i) {
 c = i;
}
int getc() {
 return c;
}
}
class AccessTest {
public static void main(String args[]) {
 Test ob = new Test();
 ob.a = 10;
 ob.b = 20;
 ob.c = 100;
 ob.setc(100);
 System.out.println("a, b, and c: " + ob.a + " " + ob.b + " " + ob.getc());
}
}
```

```
a, b, and c: 10 20 100
```

Example for private access specifier

```
class Test {
int a;
int b;
private int c;
void setc(int i) {
c = i;
}
int getc() {
return c;
}
}
class AccessTest {
public static void main(String args[]) {
Test ob = new Test();
ob.a = 10;
ob.b = 20;
ob.c = 100;
ob.setc(100);
System.out.println("a, b, and c: " + ob.a + " " + ob.b + " " + ob.getc());
}
}
```

```
AccessTest.java:18: error: c has private access in Test
ob.c = 100;
   ^
1 error
```

9. Explain static variable and static methods in JAVA.

A static variable is shared among all instances of a class. It is stored in memory once (in the class's static area) and retains its value across multiple objects. Static variables are useful for constants or shared properties.

General Form : class Example {

static int a = 10; // Static variable static int b; // Uninitialized static variable

}

}

A static method can be called without creating an object of the class. It can only access static variables and methods directly but cannot access instance (non-static) members.

General Form : class Example {

static void display() { // Static method

System.out.println("Static method called!");
}
}

```
class UseStatic {
 static int a = 3;
 static int b;
static void meth(int x) {
 System.out.println("x = " + x);
 System.out.println("a = " + a);
System.out.println("b = " + b); }
static {
 System.out.println("Static block initialized."); b = a * 4;
}
public static void main(String args[]) {
 meth(42);
}
 }
```

```
Static block initialized.
x = 42
a = 3
b = 12
```

10. Explain call by value and call by reference with an example program

There are two ways that a computer language can pass an argument to a subroutine.

The first way is call-by-value. This approach copies the value of an argument into the formal parameter of the subroutine. Therefore, changes made to the parameter of the subroutine have no effect on the argument.

```
class Test {
void meth(int i, int j) {
 i *= 2;
 j /= 2;
}
}
class CallByValue {
public static void main(String args[]) {
 Test ob = new Test();
 int a = 15, b = 20;
 System.out.println("a and b before call: " + a + " " + b);
 ob.meth(a, b);
 System.out.println("a and b after call: " + a + " " + b);
}
 }
```

```
a and b before call: 15 20
a and b after call: 15 20
```

The second way an argument can be passed is call-by-reference. In this approach, a reference to an argument (not the value of the argument) is passed to the parameter. Inside the subroutine, this reference is used to access the actual argument specified in the call. This means that changes made to the parameter will affect the argument used to call the subroutine.

```
class Test {
 int a, b;
Test(int i, int j) {
 a = i;
 b = j;
} // pass an object
void meth(Test o) {
 o.a *=  2;
 o.b /= 2;
} }
class CallByRef {
public static void main(String args[]) {
 Test ob = new Test(15, 20);
 System.out.println("ob.a and ob.b before call: " + ob.a + " " + ob.b);
 ob.meth(ob);
 System.out.println("ob.a and ob.b after call: " + ob.a + " " + ob.b);
}
 }
```

```
ob.a and ob.b before call: 15 20
ob.a and ob.b after call: 30 10
```

11. Write a JAVA program to sort list of elements in ascending and descending order.

```
Import java.util.Scanner;   public class ExchangeSort {  public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
System.out.println("Enter the number of elements in the array: ");  int n = sc.nextInt();  int[] arr = new int[n];
System.out.println("Enter the elements of the array: ");  for (int i = 0; i < n; i++) {  arr[i] = sc.nextInt();
}
for (int i = 0; i < n ; i++) {  for (int j = i + 1; j < n; j++) {  if (arr[i] > arr[j]) {  int temp = arr[i];  arr[i] = arr[j];
arr[j] = temp;
}
}
}
System.out.println("\nSorted array in ascending order: ");  for (int i = 0; i < n; i++) {
System.out.print(arr[i] + " ");
}
for (int i = 0; i < n - 1; i++) {  for (int j = i + 1; j < n; j++) {  if (arr[i] < arr[j]) {  int temp = arr[i];  arr[i] = arr[j];
arr[j] = temp;
}
}
}
System.out.println("\nSorted array in descending order: "); for (int i = 0; i < n; i++) {
System.out.print(arr[i] + " ");
```

```
}
}
}
```

```
Enter the number of elements in the array:
3
Enter the elements of the array:
5
2
7

Sorted array in ascending order:
2 5 7
Sorted array in descending order:
7 5 2
```

ACET